

Comprehensive JavaScript Notes: From Basics to Advanced

JavaScript is a versatile, high-level, interpreted programming language primarily used for creating interactive and dynamic web content. It's also used in server-side development (Node.js), mobile apps (React Native), and desktop applications (Electron).

I. Introduction to JavaScript

What is JavaScript?

- **High-level:** Abstracts away complex computer details.
- **Interpreted:** Code is executed line by line by an interpreter, not compiled beforehand.
- **Dynamic:** Supports dynamic typing (variable types are checked at runtime).
- **Multi-paradigm:** Supports object-oriented, imperative, and functional programming styles.
- **Single-threaded:** Executes one command at a time (though it handles concurrency through the event loop).

Where Does JavaScript Run?

- **Browsers:** The original environment, executing code to make web pages interactive.
- **Node.js:** A runtime environment that allows JavaScript to run on the server-side.
- **Other Environments:** Desktop apps (Electron), mobile apps (React Native), IoT devices.

Setting Up Your Environment

For browser-based JavaScript, you just need a web browser and a text editor. You can embed JavaScript directly in HTML or link external .js files.

Example: Basic HTML with JavaScript

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>My First JS Page</title>
</head>
<body>
  <h1>Hello, JavaScript!</h1>
  <script>
    // JavaScript code goes here
    console.log("This message appears in the browser console.");
    document.querySelector('h1').style.color = 'blue';
```

```

        // Try changing the color to 'red' or 'green' and see the
effect!
    </script>
</body>
</html>

```

II. JavaScript Basics

1. Variables

Variables are containers for storing data values.

- **var (Legacy):** Function-scoped. Can be re-declared and re-assigned.


```

var greeting = "Hello";
var greeting = "Hi"; // Re-declaration is allowed
greeting = "Hey"; // Re-assignment is allowed
console.log(greeting); // Output: Hey

```
- **let (ES6+):** Block-scoped. Cannot be re-declared in the same scope, but can be re-assigned.


```

let name = "Alice";
// let name = "Bob"; // Error: 'name' has already been declared
name = "Charlie"; // Re-assignment is allowed
console.log(name); // Output: Charlie

if (true) {
    let blockVar = "I'm in a block";
    console.log(blockVar); // Output: I'm in a block
}
// console.log(blockVar); // Error: blockVar is not defined
// (outside block scope)

```
- **const (ES6+):** Block-scoped. Cannot be re-declared or re-assigned. Must be initialized at declaration.


```

const PI = 3.14159;
// const PI = 3.0; // Error: Identifier 'PI' has already been
// declared
// PI = 3.0; // Error: Assignment to constant variable.
console.log(PI); // Output: 3.14159

const user = { name: "David" };
user.name = "Eve"; // Allowed: Modifying properties of an object
// declared with const
// user = { age: 30 }; // Error: Assignment to constant variable
// (re-assigning the object itself)
console.log(user); // Output: { name: "Eve" }

```

2. Data Types

JavaScript has primitive and non-primitive (object) data types.

Primitive Data Types:

- **string:** Represents textual data.

```
let message = "Hello, World!";  
let char = 'A';
```
- **number:** Represents both integer and floating-point numbers.

```
let age = 30;  
let price = 99.99;
```
- **boolean:** Represents a logical entity and can have two values: true or false.

```
let isActive = true;  
let hasPermission = false;
```
- **undefined:** Represents a variable that has been declared but not yet assigned a value.

```
let score;  
console.log(score); // Output: undefined
```
- **null:** Represents the intentional absence of any object value.

```
let car = null;  
console.log(car); // Output: null
```
- **symbol (ES6+):** Represents a unique identifier.

```
const id1 = Symbol('id');  
const id2 = Symbol('id');  
console.log(id1 === id2); // Output: false
```
- **bigint (ES2020):** Represents integers with arbitrary precision.

```
const largeNumber = 1234567890123456789012345678901234567890n;  
console.log(largeNumber);
```

Non-Primitive Data Type:

- **object:** A collection of properties, where each property has a name (key) and a value.
 - **Objects:**

```
let person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 30,  
  isStudent: false  
};  
console.log(person.firstName); // Output: John  
console.log(person['age']); // Output: 30
```
 - **Arrays (special type of object):** Ordered lists of values.

```
let colors = ["red", "green", "blue"];
```

```
console.log(colors[0]); // Output: red
console.log(colors.length); // Output: 3
```

- **Functions** (special type of object): Blocks of reusable code.

```
function greet() {
    console.log("Hello!");
}
console.log(typeof greet); // Output: function (which is a
sub-type of object)
```

3. Operators

Operators perform operations on values and variables.

- **Arithmetic Operators:** +, -, *, /, % (modulus), ** (exponentiation), ++ (increment), -- (decrement)

```
let a = 10, b = 3;
console.log(a + b); // 13
console.log(a - b); // 7
console.log(a ** b); // 1000 (10 to the power of 3)
a++;
console.log(a); // 11
```
- **Assignment Operators:** =, +=, -=, *=, /=, %=, **=

```
let x = 5;
x += 3; // x is now 8 (x = x + 3)
console.log(x); // Output: 8
```
- **Comparison Operators:** == (loose equality), === (strict equality), !=, !==, >, <, >=, <=

```
console.log(5 == '5'); // true (loose equality, type coercion)
console.log(5 === '5'); // false (strict equality, no type coercion)
console.log(10 > 5); // true
```
- **Logical Operators:** && (AND), || (OR), ! (NOT)

```
let sunny = true;
let warm = false;
console.log(sunny && warm); // false
console.log(sunny || warm); // true
console.log(!sunny); // false
```
- **Ternary Operator:** condition ? exprIfTrue : exprIfFalse

```
let age = 18;
let status = (age >= 18) ? "Adult" : "Minor";
console.log(status); // Output: Adult
```

4. Control Flow

Statements that control the order in which code is executed.

- **if...else if...else:** Executes a block of code based on a condition.

```
let temperature = 25;  
if (temperature > 30) {  
  console.log("It's hot!");  
} else if (temperature > 20) {  
  console.log("It's warm.");  
} else {  
  console.log("It's cold.");  
}
```
- **switch:** Evaluates an expression and executes code based on matching case values.

```
let day = "Monday";  
switch (day) {  
  case "Monday":  
    console.log("Start of the week.");  
    break;  
  case "Friday":  
    console.log("Almost weekend!");  
    break;  
  default:  
    console.log("Mid-week or weekend day.");  
}
```

5. Loops

Repeatedly execute a block of code.

- **for loop:** Executes a block of code a specified number of times.

```
for (let i = 0; i < 5; i++) {  
  console.log("Count:", i);  
}  
// Output:  
// Count: 0  
// Count: 1  
// Count: 2  
// Count: 3  
// Count: 4
```
- **while loop:** Executes a block of code as long as a specified condition is true.

```
let count = 0;  
while (count < 3) {  
  console.log("While count:", count);  
  count++;  
}
```

```
// Output:
// While count: 0
// While count: 1
// While count: 2
```

- **do...while loop:** Similar to while, but the block of code is executed at least once before the condition is tested.

```
let i = 0;
do {
  console.log("Do-while count:", i);
  i++;
} while (i < 0); // Condition is false, but runs once
// Output:
// Do-while count: 0
```

- **for...in loop:** Iterates over the enumerable properties of an object.

```
const car = { brand: "Toyota", model: "Camry", year: 2020 };
for (let key in car) {
  console.log(`${key}: ${car[key]}`);
}
// Output:
// brand: Toyota
// model: Camry
// year: 2020
```

- **for...of loop (ES6+):** Iterates over iterable objects (like Arrays, Strings, Maps, Sets, etc.).

```
const colors = ["red", "green", "blue"];
for (let color of colors) {
  console.log(color);
}
// Output:
// red
// green
// blue
```

6. Functions

Blocks of reusable code that perform a specific task.

- **Function Declaration:** Defined using the function keyword. Hoisted to the top of their scope.

```
function add(a, b) {
  return a + b;
}
console.log(add(5, 3)); // Output: 8
```

- **Function Expression:** Defined as part of an expression (e.g., assigned to a variable). Not hoisted.

```
const multiply = function(a, b) {  
    return a * b;  
};  
console.log(multiply(4, 2)); // Output: 8
```

- **Arrow Functions (ES6+):** A more concise way to write function expressions. They do not have their own this context.

```
const subtract = (a, b) => a - b;  
console.log(subtract(10, 4)); // Output: 6
```

```
const greetUser = name => console.log(`Hello, ${name}!`);  
greetUser("Alice"); // Output: Hello, Alice!
```

III. Intermediate JavaScript

1. Arrays

Arrays are ordered collections of data. They come with many built-in methods.

- **push():** Adds one or more elements to the end of an array.

```
let fruits = ["apple", "banana"];  
fruits.push("cherry");  
console.log(fruits); // ["apple", "banana", "cherry"]
```
- **pop():** Removes the last element from an array and returns that element.

```
let lastFruit = fruits.pop();  
console.log(fruits); // ["apple", "banana"]  
console.log(lastFruit); // "cherry"
```
- **shift():** Removes the first element from an array and returns that element.

```
let firstFruit = fruits.shift();  
console.log(fruits); // ["banana"]  
console.log(firstFruit); // "apple"
```
- **unshift():** Adds one or more elements to the beginning of an array.

```
fruits.unshift("grape", "kiwi");  
console.log(fruits); // ["grape", "kiwi", "banana"]
```
- **splice(start, deleteCount, item1, ...):** Changes the contents of an array by removing or replacing existing elements and/or adding new elements in place.

```
let numbers = [1, 2, 3, 4, 5];  
numbers.splice(2, 1); // Remove 1 element at index 2 (removes 3)  
console.log(numbers); // [1, 2, 4, 5]  
numbers.splice(1, 0, 99); // Add 99 at index 1, delete 0 elements  
console.log(numbers); // [1, 99, 2, 4, 5]  
numbers.splice(3, 2, 100, 101); // Replace 2 elements from index 3  
with 100, 101
```

```
console.log(numbers); // [1, 99, 2, 100, 101]
```

- **slice(start, end):** Returns a shallow copy of a portion of an array into a new array object. Does not modify the original array.

```
let original = [1, 2, 3, 4, 5];  
let sliced = original.slice(1, 4); // Elements from index 1 up to  
    (but not including) 4  
console.log(sliced); // [2, 3, 4]  
console.log(original); // [1, 2, 3, 4, 5] (original unchanged)
```

- **map():** Creates a new array populated with the results of calling a provided function on every element in the calling array.

```
const nums = [1, 2, 3];  
const doubled = nums.map(num => num * 2);  
console.log(doubled); // [2, 4, 6]
```

- **filter():** Creates a new array with all elements that pass the test implemented by the provided function.

```
const ages = [12, 19, 25, 8];  
const adults = ages.filter(age => age >= 18);  
console.log(adults); // [19, 25]
```

- **reduce():** Executes a reducer function on each element of the array, resulting in a single output value.

```
const numbersToSum = [1, 2, 3, 4];  
const sum = numbersToSum.reduce((accumulator, currentValue) =>  
    accumulator + currentValue, 0);  
console.log(sum); // 10
```

- **forEach():** Executes a provided function once for each array element. Does not return a new array.

```
const names = ["Anna", "Ben", "Chris"];  
names.forEach(name => console.log(`Hello, ${name}`));  
// Output:  
// Hello, Anna  
// Hello, Ben  
// Hello, Chris
```

2. Objects

Objects are fundamental for grouping related data and functionality.

- **Properties and Methods:**

```
let user = {  
    firstName: "Jane",  
    lastName: "Doe",  
    age: 25,  
    // Method
```



```

    fullName: function() {
        return `${this.firstName} ${this.lastName}`;
    },
    greet() { // Shorthand method syntax (ES6+)
        console.log(`Hello, my name is ${this.fullName()}.`);
    }
};
console.log(user.firstName); // Access property
console.log(user['lastName']); // Access property using bracket notation
console.log(user.fullName()); // Call method
user.greet();

```

- **this** keyword: Refers to the object that is currently executing the function. Its value depends on how the function is called.
 - In a method, **this** refers to the object the method belongs to.
 - In a regular function call (non-strict mode), **this** refers to the global object (window in browsers, global in Node.js).
 - In strict mode, **this** in a regular function call is undefined.
 - Arrow functions do not have their own **this**; they inherit **this** from the enclosing lexical context.
 - **Controlling this:** Methods like `call()`, `apply()`, and `bind()` can explicitly set the **this** context of a function.

```

function introduce(city, job) {
    console.log(`My name is ${this.firstName} and I live in ${city}. I am a ${job}.`);
}
const person = { firstName: "Alice" };

```

```

introduce.call(person, "New York", "Developer"); // Output:
My name is Alice and I live in New York. I am a Developer.
introduce.apply(person, ["London", "Designer"]); // Output:
My name is Alice and I live in London. I am a Designer.

```

```

const introduceAlice = introduce.bind(person, "Paris");
introduceAlice("Engineer"); // Output: My name is Alice and I
live in Paris. I am an Engineer.

```

- **Object.keys():** Returns an array of a given object's own enumerable property names.

```
console.log(Object.keys(user)); // ["firstName", "lastName", "age", "fullName", "greet"]
```
- **Object.values():** Returns an array of a given object's own enumerable property values.

```
console.log(Object.values(user)); // ["Jane", "Doe", 25, f, f]
```

 (function references)
- **Object.entries():** Returns an array of a given object's own enumerable string-keyed property [key, value] pairs.

```

console.log(Object.entries(user));
// [
//   ["firstName", "Jane"],
//   ["lastName", "Doe"],
//   ["age", 25],
//   // ... and function entries
// ]

```

3. Scope

Scope determines the accessibility of variables, functions, and objects in some part of your code.

- **Global Scope:** Variables declared outside any function or block are globally scoped and accessible from anywhere.

```

let globalVar = "I'm global";
function accessGlobal() {
    console.log(globalVar); // Accessible
}
accessGlobal();

```

- **Function Scope:** Variables declared with `var` inside a function are function-scoped and only accessible within that function.

```

function myFunction() {
    var funcVar = "I'm function-scoped";
    console.log(funcVar);
}
myFunction();
// console.log(funcVar); // Error: funcVar is not defined

```

- **Block Scope (ES6+):** Variables declared with `let` or `const` inside a block (`{}`) are block-scoped and only accessible within that block.

```

if (true) {
    let blockLet = "I'm block-scoped (let)";
    const blockConst = "I'm block-scoped (const)";
    console.log(blockLet);
    console.log(blockConst);
}
// console.log(blockLet); // Error: blockLet is not defined
// console.log(blockConst); // Error: blockConst is not defined

```

4. Hoisting

JavaScript's default behavior of moving declarations to the top of the current scope (global or function scope) before code execution.

- **Variable Hoisting (var):** Only declarations are hoisted, not initializations.

```

console.log(hoistedVar); // Output: undefined (declaration is

```

```

hoisted)
var hoistedVar = "Hello";
console.log(hoistedVar); // Output: Hello

```

- **Variable Hoisting (let, const):** Also hoisted, but they are in a "Temporal Dead Zone" (TDZ) until their actual declaration is encountered. Accessing them before declaration results in a `ReferenceError`.

```

// console.log(hoistedLet); // Error: Cannot access 'hoistedLet'
// before initialization
let hoistedLet = "Hello";

```

- **Function Hoisting:** Function declarations are fully hoisted (both declaration and definition).

```

sayHello(); // Output: Hello! (function is hoisted)
function sayHello() {
    console.log("Hello!");
}

```

- **Function Expressions/Arrow Functions:** Not hoisted in the same way; they behave like variable hoisting.

```

// sayGoodbye(); // Error: sayGoodbye is not a function (if using
// var) or ReferenceError (if using let/const)
const sayGoodbye = function() {
    console.log("Goodbye!");
};
sayGoodbye();

```

5. Closures

A closure is the combination of a function bundled together (enclosed) with references to its surrounding state (the lexical environment). In simpler terms, a closure gives you access to an outer function's scope from an inner function, even after the outer function has finished executing.

```

function outerFunction(outerVariable) {
    return function innerFunction(innerVariable) {
        console.log('Outer Variable:', outerVariable);
        console.log('Inner Variable:', innerVariable);
    };
}

const newFunction = outerFunction('I am outer');
newFunction('I am inner');
// Output:
// Outer Variable: I am outer
// Inner Variable: I am inner
// Even though outerFunction has finished executing, newFunction (the
// inner function)
// still "remembers" and can access outerVariable.

```

6. Prototypes and Prototypal Inheritance

JavaScript uses prototypal inheritance. Every object has a prototype object, from which it inherits methods and properties.

- **Prototype Chain:** When you try to access a property or method on an object, JavaScript first looks for it on the object itself. If not found, it looks on the object's prototype, then on the prototype's prototype, and so on, until it reaches null.
- `__proto__`: A common (though deprecated for direct use) way to access an object's prototype.
- `Object.getPrototypeOf()`: The standard way to get an object's prototype.

```
const animal = {
  eats: true,
  walk() {
    console.log("Animal walks.");
  }
};
const rabbit = {
  jumps: true,
  __proto__: animal // rabbit inherits from animal
};
const longEar = {
  earLength: 10,
  __proto__: rabbit // longEar inherits from rabbit
};
console.log(longEar.eats); // true (inherited from animal via rabbit)
longEar.walk(); // Animal walks. (inherited from animal)
console.log(longEar.jumps); // true (inherited from rabbit)
```

7. new keyword and Constructor Functions

Constructor functions are regular functions used with the new keyword to create new objects. When a function is called with new:

1. A new empty object is created.
2. `this` in the constructor function points to this new object.
3. The prototype of the new object is set to `ConstructorFunction.prototype`.
4. The constructor function executes, populating the new object.
5. The new object is returned (unless the constructor explicitly returns another object).

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}
// Add a method to the Person's prototype
Person.prototype.greet = function() {
  console.log(`Hello, my name is ${this.name} and I am ${this.age}
years old.`);
}
```

```
};
const person1 = new Person("Alice", 30);
const person2 = new Person("Bob", 25);
person1.greet(); // Output: Hello, my name is Alice and I am 30 years old.
person2.greet(); // Output: Hello, my name is Bob and I am 25 years old.
console.log(person1.__proto__ === Person.prototype); // true
```

IV. Advanced JavaScript

1. Asynchronous JavaScript

JavaScript is single-threaded, meaning it executes one task at a time. Asynchronous operations allow long-running tasks (like network requests, timers) to run in the background without blocking the main thread.

- **Callbacks (Traditional):** Functions passed as arguments to other functions, to be executed later when an asynchronous operation completes. Can lead to "callback hell" (deeply nested callbacks).

```
function fetchData(callback) {
    setTimeout(() => {
        const data = "Data fetched!";
        callback(data);
    }, 1000); // Simulate network delay
}
fetchData(function(result) {
    console.log(result); // Output: Data fetched! (after 1 second)
});
console.log("Fetching data..."); // This runs immediately
```

- **Promises (ES6+):** Objects representing the eventual completion or failure of an asynchronous operation and its resulting value. They provide a cleaner way to handle asynchronous code.
 - **States:** pending, fulfilled (resolved), rejected.
 - **Methods:** .then() (for success), .catch() (for error), .finally() (always executed).

```
function fetchDataPromise() {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            const success = true; // Simulate success/failure
            if (success) {
                resolve("Data fetched successfully!");
            } else {
                reject("Failed to fetch data.");
            }
        }, 1000);
    });
}
```

```

    }
    fetchDataPromise()
      .then(data => {
        console.log(data); // Output: Data fetched successfully!
      })
      .catch(error => {
        console.error(error);
      })
      .finally(() => {
        console.log("Promise finished.");
      });
    console.log("Initiating promise...");

```

- **Async/Await (ES2017):** Syntactic sugar built on top of Promises, making asynchronous code look and behave more like synchronous code.
 - **async keyword:** Used to define an asynchronous function. An async function implicitly returns a Promise.
 - **await keyword:** Can only be used inside an async function. It pauses the execution of the async function until the Promise it's waiting for settles (resolves or rejects).

```

async function getUserData() {
  try {
    console.log("Fetching user data...");
    const response = await fetchDataPromise(); // Await the
promise
    console.log("User data:", response);
    return response;
  } catch (error) {
    console.error("Error fetching user data:", error);
    throw error; // Re-throw to propagate the error
  } finally {
    console.log("getUserData function finished.");
  }
}

getUserData();
console.log("Program continues...");

```

2. Error Handling

Using try...catch...finally blocks to handle errors gracefully.

- **try block:** Contains the code that might throw an error.
- **catch block:** Executes if an error occurs in the try block. It receives the error object as an argument.
- **finally block:** Executes regardless of whether an error occurred or not. Often used for cleanup.

```

function divide(a, b) {
  if (b === 0) {
    throw new Error("Division by zero is not allowed."); // Throw

```

```

a custom error
    }
    return a / b;
}
try {
    let result1 = divide(10, 2);
    console.log("Result 1:", result1); // Output: Result 1: 5
    let result2 = divide(10, 0); // This will throw an error
    console.log("Result 2:", result2); // This line will not be
reached
} catch (error) {
    console.error("An error occurred:", error.message); // Output: An
error occurred: Division by zero is not allowed.
} finally {
    console.log("Division attempt finished."); // Always runs
}
console.log("Program continues after error handling.");

```

3. ES6+ Features (Modern JavaScript)

ECMAScript 2015 (ES6) introduced many significant features that have become standard practice.

- **Destructuring Assignment:** A convenient way to extract values from arrays or properties from objects into distinct variables.
 - **Array Destructuring:**

```

const [first, second, ...rest] = [10, 20, 30, 40, 50];
console.log(first); // 10
console.log(second); // 20
console.log(rest); // [30, 40, 50]

```
 - **Object Destructuring:**

```

const { name, age } = { name: "Sara", age: 28, city: "NY" };
console.log(name); // Sara
console.log(age); // 28
const { city: hometown, age: personAge } = { name: "Sara",
age: 28, city: "NY" };
console.log(hometown); // NY
console.log(personAge); // 28

```
- **Spread (...) and Rest (...) Operators:**
 - **Spread Operator:** Expands an iterable (like an array or string) into individual elements. Useful for copying arrays/objects, merging, or passing arguments.


```

const arr1 = [1, 2];
const arr2 = [3, 4];
const combinedArray = [...arr1, ...arr2]; // [1, 2, 3, 4]
console.log(combinedArray);

```

```

const obj1 = { a: 1, b: 2 };
const obj2 = { c: 3, d: 4 };
const combinedObject = { ...obj1, ...obj2 }; // { a: 1, b: 2,
c: 3, d: 4 }
console.log(combinedObject);

function sumAll(x, y, z) {
    return x + y + z;
}
const numbers = [1, 2, 3];
console.log(sumAll(...numbers)); // 6 (spreads array elements
as arguments)

```

- **Rest Parameter:** Collects an indefinite number of arguments into an array. Must be the last parameter in a function definition.

```

function collectArgs(firstArg, ...remainingArgs) {
    console.log(firstArg); // "A"
    console.log(remainingArgs); // ["B", "C", "D"]
}
collectArgs("A", "B", "C", "D");

```

- **Classes:** Provide a cleaner, more object-oriented syntax for creating objects and handling inheritance, though they are still prototypal inheritance under the hood.

```

class Animal {
    constructor(name) {
        this.name = name;
    }
    speak() {
        console.log(`${this.name} makes a sound.`);
    }
}

class Dog extends Animal { // Inheritance using 'extends'
    constructor(name, breed) {
        super(name); // Call parent class constructor
        this.breed = breed;
    }
    speak() {
        console.log(`${this.name} (${this.breed}) barks.`);
    }
    fetch() {
        console.log(`${this.name} fetches the ball.`);
    }
}

const myAnimal = new Animal("Generic Animal");
myAnimal.speak(); // Output: Generic Animal makes a sound.
const myDog = new Dog("Buddy", "Golden Retriever");
myDog.speak(); // Output: Buddy (Golden Retriever) barks.
myDog.fetch(); // Output: Buddy fetches the ball.

```


- **Modules (ES Modules):** A way to organize JavaScript code into separate files, allowing for better code organization, reusability, and dependency management.
 - **export:** Used to export functions, objects, or primitive values from a module.
 - **import:** Used to import exported members from another module.

math.js (Module to export)// math.js

```
export const PI = 3.14159;
export function add(a, b) {
  return a + b;
}
export function subtract(a, b) {
  return a - b;
}
// Default export (only one per module)
export default class Calculator {
  multiply(a, b) {
    return a * b;
  }
}
```

app.js (Module to import)// app.js

```
import { PI, add } from './math.js'; // Named imports
import Calculator from './math.js'; // Default import
```

```
console.log(PI); // 3.14159
console.log(add(5, 2)); // 7
const calc = new Calculator();
console.log(calc.multiply(4, 3)); // 12
```

Note: Modules require a server environment (e.g., a simple HTTP server or bundling tool like Webpack, Rollup, or Parcel) to run in browsers, or Node.js. Bundlers are often used in production to optimize module loading and performance.

4. Event Loop (Brief Explanation)

The Event Loop is a crucial concept for understanding how JavaScript handles asynchronous operations.

- **Call Stack:** Where synchronous code is executed. Functions are pushed onto the stack when called and popped off when they return.
- **Web APIs / Node.js APIs:** Browser/Node.js provided APIs (e.g., setTimeout, fetch, DOM events) that handle asynchronous tasks. When these are called, they are moved off the call stack and handled by the environment.
- **Callback Queue (Task Queue / Macrotask Queue):** When an asynchronous operation completes (e.g., setTimeout finishes, fetch response arrives), its callback function is placed in the callback queue.
- **Event Loop:** Continuously monitors the Call Stack and the Callback Queue. If the Call Stack is empty, it takes the first function from the Callback Queue and pushes it onto the Call Stack for execution.
- **Microtask Queue:** (e.g., Promises' .then(), .catch(), await resolutions) has higher priority than the Callback Queue. Microtasks are executed after the current script finishes and

before the next macrotask is picked up.
This mechanism allows JavaScript to appear non-blocking despite being single-threaded.

5. DOM Manipulation (Browser-specific)

The Document Object Model (DOM) is a programming interface for web documents. It represents the page structure as a tree of objects, allowing JavaScript to interact with HTML and CSS.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>DOM Example</title>
</head>
<body>
  <h1 id="myHeading">Hello, DOM!</h1>
  <button id="myButton">Change Text</button>
  <div class="container">
    <p>This is a paragraph.</p>
  </div>
  <script>
    // Get elements by ID
    const heading = document.getElementById('myHeading');
    console.log(heading.textContent); // Output: Hello, DOM!

    // Change text content
    heading.textContent = "DOM Manipulated!";
    heading.style.color = "purple"; // Change style

    // Get elements by class name (returns a NodeList)
    const containers =
document.getElementsByClassName('container');
    if (containers.length > 0) {
      containers[0].style.backgroundColor = '#f0f0f0';
    }

    // Query Selector (first matching element)
    const paragraph = document.querySelector('.container p');
    paragraph.style.fontWeight = 'bold';

    // Query Selector All (all matching elements, returns a
NodeList)
    const allParagraphs = document.querySelectorAll('p');
    allParagraphs.forEach(p => {
      p.style.border = '1px solid lightgray';
    });
```

```

    // Event Listener
    const button = document.getElementById('myButton');
    button.addEventListener('click', () => {
        // In a real application, you'd use a custom message box
        // or modal instead of alert()
        console.log("Button clicked!");
        heading.textContent = "Button was clicked!";
    });

    // Creating new elements
    const newDiv = document.createElement('div');
    newDiv.textContent = "I am a new div!";
    document.body.appendChild(newDiv); // Add to the end of the
body
    </script>
</body>
</html>

```

6. Fetch API (Browser-specific)

The Fetch API provides an interface for fetching resources (e.g., across the network). It's a modern replacement for XMLHttpRequest and uses Promises.

```

async function fetchPosts() {
    try {
        const response = await
fetch('https://jsonplaceholder.typicode.com/posts/1');
        // Check if the request was successful (status code 200-299)
        if (!response.ok) {
            throw new Error(`HTTP error! status: ${response.status}`);
        }
        const data = await response.json(); // Parse JSON response
        console.log("Fetched Post:", data);
    } catch (error) {
        console.error("Error fetching post:", error);
    }
}
fetchPosts();

// Example with POST request
async function createPost() {
    try {
        const response = await
fetch('https://jsonplaceholder.typicode.com/posts', {
            method: 'POST', // Specify the method
            headers: {
                'Content-Type': 'application/json', // Indicate JSON

```

body

```
    },
    body: JSON.stringify({ // Convert object to JSON string
      title: 'foo',
      body: 'bar',
      userId: 1,
    }),
  });
  if (!response.ok) {
    throw new Error(`HTTP error! status: ${response.status}`);
  }
  const data = await response.json();
  console.log("Created Post:", data);
} catch (error) {
  console.error("Error creating post:", error);
}
}
createPost();
```

This comprehensive overview covers the fundamental to advanced aspects of JavaScript, providing a solid foundation for your development journey. Remember that practice is key to mastering these concepts!